

Documento de Justificación para el modelado UML del sistema SISI (Sistema de Seguimiento de Indicadores de Salud Infantil)

Autor: Dorantes Rodríguez Ulick

Contenido

Justificación.....	3
Jerarquía de Herencia 'Persona' -> 'PersonalMedico' -> 'Doctor'	3
Clase intermedia 'Consulta' y su relación de Composición con 'PacientePediatico' y relación de agregación con 'Doctor'	3
Clase abstracta 'Medicion' y sus clases herederas	4
Relación de la clase 'GestorUmbrables' con la clase abstracta 'Medicion'	5
Clase 'GestorConsultas' y su relación con la clase intermedia 'Consulta'	5
Existencia de la clase 'AnalizadorEstadistico' y su relación débil con la clase ' PacientePediatico'	6
Justificación de la clase 'App' y su relación de composición con todas las vistas.....	6
Relación de dependencia de 'VistaReporte' con la clase 'AnalizadorEstadistico'	7

Justificación

Jerarquía de Herencia 'Persona' -> 'PersonalMedico' -> 'Doctor'

Este diseño acata directamente el requerimiento de 'Implementar al menos dos niveles de herencia efectiva'. En lugar de forzar esta relación artificialmente, se modeló una jerarquía natural y semánticamente correcta: el **Nivel 0** (Persona) establece la base, el **Nivel 1** (PersonalMedico) añade el contexto profesional, y el **Nivel 2** (Doctor) concreta el rol operativo.

La separación en tres clases garantiza que cada nivel de la jerarquía tenga un único motivo para cambiar, delimitando claramente los contextos de los datos:

Persona (Clase Abstracta): Encapsula exclusivamente la identidad civil y biológica (nombre, sexo, CURP). Su responsabilidad es proveer una base genérica para cualquier ser humano que interactúe con el sistema (aplicable también para los pacientes infantiles).

PeronalMedico (Clase Abstracta): Encapsula la identidad legal y profesional. Su responsabilidad es manejar atributos compartidos por los trabajadores de la salud (como la cédula) y definir contratos de comportamiento (como la validación de credenciales).

Doctor (Clase Concreta): Encapsula el contexto transaccional y operativo. Su responsabilidad es manejar la agenda diaria, el consultorio físico y la relación directa con el historial de los pacientes.

La existencia de la clase intermedia PersonalMedico facilita la implementación de barreras de seguridad polimórficas en la capa de repositorios y vistas. Permite evaluar permisos mediante validaciones de tipo antes de otorgar acceso de escritura a los historiales clínicos de los pacientes, asegurando que solo personal autorizado pueda modificar indicadores de salud.

Clase intermedia 'Consulta' y su relación de Composición con 'PacientePediatico' y relación de agregación con 'Doctor'

Al crear la clase intermedia Consulta, se transforma un evento transaccional en un objeto de primera clase. Esto permite encapsular atributos propios del evento (como la fecha, y en un futuro, el diagnóstico o el tratamiento) sin contaminar las clases biológicas puras (PacientePediatico o Doctor).

Relación de composición: El historial médico le pertenece integral y solamente al paciente. Una Consulta no tiene razón de existir en el sistema como una entidad aislada; es un fragmento del expediente del niño. Si un PacientePediatico es eliminado o purgado de la base de datos, todos sus registros de consulta locales pierden su contexto principal y deben ser destruidos en cascada.

Relación de agregación: Un doctor participa en una consulta, la "atiende" o la "supervisa", pero no es dueño exclusivo de la existencia de ese registro clínico de la misma forma que el expediente del paciente. Si un Doctor renuncia, es dado de baja o eliminado del sistema activo, los registros de las consultas que atendió en el pasado deben permanecer intactos por motivos de auditoría y continuidad médica.

Este diseño sienta las bases para el patrón arquitectónico. Al aislar la Consulta, el sistema permite inyectar fácilmente gestores intermedios (como el GestorConsultas) que puedan filtrar, buscar y cruzar datos (por ejemplo, buscar todas las consultas de un mes específico) operando sobre una lista de objetos estandarizados, en lugar de tener que iterar y desanidar listas complejas dentro de cada paciente y cada doctor.

Clase abstracta 'Medicion' y sus clases herederas

La decisión de modelar los indicadores de salud utilizando una clase base abstracta (Medicion) de la cual derivan clases concretas específicas (MedicionTemperatura, MedicionPeso, etc.) constituye el núcleo del motor polimórfico del sistema.

No existe una "medición" genérica flotando en el vacío; siempre se mide una magnitud física específica (grados, kilogramos, latidos). Al declarar Medicion como una Clase Base Abstracta (ABC), se bloquea explícitamente la instanciación de objetos sin contexto. El sistema obliga a nivel de compilación a que todo dato biométrico ingresado tenga una identidad clínica concreta, evitando estados inválidos o registros de los que se desconozca su naturaleza matemática.

El uso del decorador `@abstractmethod` en la función `en_riesgo(gestor: GestorUmbrales)` -> `bool` establece un contrato de diseño inquebrantable. Al no proveer una implementación en la clase padre, se garantiza que cualquier nueva clase hija esté forzada a declarar cómo se evalúa su propio riesgo.

Si en lugar de herencia se hubiera utilizado una sola clase Medicion con un atributo tipo cada nuevo indicador médico requeriría modificar el código existente, aumentando el riesgo de introducir *bugs*. Con esta jerarquía estructurada, para agregar un nuevo indicador biométrico en el futuro, basta con crear una nueva clase derivada sin tocar ni una sola línea de la lógica preexistente en los repositorios o analizadores.

Esta jerarquía permite que los módulos de persistencia y análisis traten a todos los objetos como si fueran simplemente instancias de Medicion. El AnalizadorEstadistico y la VistaMediciones pueden recibir una lista mixta (temperaturas, pesos, tallas), iterar sobre ella y llamar al método `en_riesgo()` sin necesidad de saber qué clase específica están manipulando en ese instante. Cada objeto, gracias al polimorfismo dinámico, sabe responder correctamente según su propia naturaleza geométrica o biológica.

Relación de la clase 'GestorUmbrales' con la clase abstracta 'Medicion'

La incorporación de la clase GestorUmbrales y su vinculación directa con la abstracción principal de los indicadores médicos es una decisión de diseño orientada a la mantenibilidad, la configuración en tiempo de ejecución y la correcta separación de responsabilidades (SRP).

Si las reglas de validación se hubieran programado directamente dentro de las clases derivadas como MedicionTemperatura, se habrían introducido "números mágicos" en el código fuente. Esto impediría que los médicos o administradores del sistema ajustaran los límites de riesgo sin la intervención de un programador. El GestorUmbrales actúa como un repositorio en memoria para estas reglas de negocio, permitiendo que los límites muten dinámicamente durante la ejecución del programa.

Bajo los principios de un diseño limpio, una entidad biológica (la medición) no debería ser la responsable de conocer las políticas de salud pública o los estándares clínicos de un hospital:

Medicion (y sus hijas): Su única responsabilidad es mantener la integridad de su estado interno (qué valor numérico se midió y en qué marca de tiempo).

GestorUmbrales: Su única responsabilidad es conocer, almacenar y proveer los límites clínicamente aceptables para cada tipo de indicador biométrico.

Relacionar el gestor con la clase padre en lugar de trazar líneas individuales hacia cada clase hija (MedicionPeso, MedicionTalla, etc.) reduce drásticamente el acoplamiento y el "ruido visual" del modelo.

Al definir que la clase abstracta exija el GestorUmbrales como parámetro, se fuerza a todas las clases derivadas a utilizar esta dependencia para cumplir con su implementación de `en_riesgo()`. Cuando la interfaz gráfica itera sobre el historial de un paciente, simplemente le pasa la instancia única del gestor a cada medición. Mediante polimorfismo dinámico, cada objeto de medición interactúa con el gestor entregándole su propio tipo y su valor, recibiendo a cambio un veredicto booleano de manera limpia y sin estructuras condicionales.

Clase 'GestorConsultas' y su relación con la clase intermedia 'Consulta'

La existencia de la clase GestorConsultas responde a la necesidad de implementar el **Patrón de Diseño de Servicios (Service Layer)**. Actúa como un motor orquestador cuya única finalidad es procesar, filtrar y cruzar los datos transaccionales del hospital, separando las consultas complejas de la lógica de almacenamiento base.

Dado que la arquitectura implementó una clase intermedia (Consulta) para evitar la redundancia, los datos ahora están normalizados. El GestorConsultas nace como el motor lógico encargado de recorrer esta estructura normalizada, extraer las coincidencias (buscando por la cédula del doctor) y devolver listas limpias de objetos

PacientePediatico, resolviendo el requerimiento sin que otras clases tengan que entender la topología de los datos.

Si la lógica de filtrado de agendas médicas y doctores se hubiera programado dentro del RepositorioPacientes, dicha clase habría violado el SRP, ya que terminaría gestionando no solo el ciclo de vida de los niños, sino también las reglas de negocio del personal médico. Al delegar esta tarea al GestorConsultas, el Repositorio mantiene una alta cohesión

El diseño de este gestor actúa como un escudo o "fachada" protectora para la capa visual (app.py). La clase VistaConsultasDoctor en Tkinter no necesita saber cómo iterar sobre los historiales clínicos ni implementar estructuras tipo set para evitar pacientes duplicados.

Existencia de la clase 'AnalizadorEstadistico' y su relación débil con la clase 'PacientePediatico'

Esta clase materializa el requerimiento, consolidando todas las fórmulas y cálculos matemáticos en un solo lugar, actuando como un motor de análisis dedicado.

El modelo biológico (PacientePediatico) solo debe encargarse de mantener su estado y su historial, no de saber cómo procesar estadística matemática. Por su parte, la interfaz gráfica (VistaReporte) solo debe saber cómo dibujar tablas y textos, no cómo calcular varianzas. Extraer esta lógica a un AnalizadorEstadistico garantiza una alta cohesión: si en el futuro cambian las reglas estadísticas (ej. cambiar de desviación estándar poblacional a muestral), el único módulo que se modificará será este, dejando intactas las vistas y la base de datos.

El analizador no es dueño del paciente, ni requiere retenerlo permanentemente en su estado para funcionar a lo largo de toda la sesión del programa.

Esta decisión de diseño impacta directamente en el rendimiento de la aplicación. Al mantener la relación como una dependencia de uso efímera, el AnalizadorEstadistico se comporta como un servicio *stateless*. La interfaz gráfica lo instancia únicamente en el momento exacto en que el doctor hace clic en "Generar Reporte". Una vez que el análisis termina y se muestran los datos, el recolector de basura (*Garbage Collector*) de Python destruye el objeto analizador, liberando la memoria inmediatamente.

Justificación de la clase 'App' y su relación de composición con todas las vistas.

La implementación de la clase App como el nodo central de la interfaz gráfica y la definición de una relación de **Composición** hacia todas las vistas del sistema. fundamentan la arquitectura bajo el patrón de **Controlador Centralizado (Mediator/Controller)**.

Las vistas del sistema son componentes especializados de la interfaz de usuario (subclases de `tk.Frame`) que solo cobran sentido dentro de la ventana principal de la aplicación (`tk.Tk`). Si la clase `App` se cierra o se destruye en la memoria del sistema operativo, todas las vistas subordinadas deben ser destruidas de forma inmediata y automática en cascada. La composición impide la existencia de "vistas huérfanas" en la memoria RAM, garantizando una gestión limpia de los recursos de hardware.

No es lógicamente posible ni arquitectónicamente viable que una pantalla específica, como `VistaMediciones`, sea compartida o controlada simultáneamente por dos aplicaciones o ventanas principales distintas. Al modelar esta relación como composición con cardinalidad 1 a 1 para cada tipo de pantalla, se blinda el flujo visual, asegurando que la navegación y el intercambio de contextos permanezcan encapsulados bajo un único hilo de ejecución visual.

Al componer todas las pantallas dentro del objeto principal, se facilita la implementación de una arquitectura limpia para aplicaciones multiventana en `tkinter` sin requerir código procedural suelto.

Relación de dependencia de 'VistaReporte' con la clase 'AnalizadorEstadistico'

La interacción entre la `VistaReporte` (capa de presentación) y el `AnalizadorEstadistico` (capa de lógica de negocio) representa el punto culminante de la separación de preocupaciones en el sistema.

La clase `VistaReporte` tiene una única responsabilidad: renderizar píxeles en la pantalla (dibujar tablas, etiquetas y botones). No debe conocer las fórmulas de la desviación estándar ni iterar sobre historiales clínicos. Al delegar esta tarea al `AnalizadorEstadistico`, se garantiza que si las fórmulas matemáticas cambian, el código visual de `tkinter` no sufrirá ninguna alteración, respetando el Principio de Responsabilidad Única (SRP).

La dependencia de uso indica que la vista instancia al `AnalizadorEstadistico` **únicamente** dentro del método `_generar_y_mostrar_estadisticas()` en el momento exacto en que el usuario hace clic en el botón. El analizador nace, procesa los datos, entrega el reporte y su ciclo de vida termina inmediatamente.

El `AnalizadorEstadistico` actúa como un adaptador. Convierte la complejidad de los objetos biométricos (`MedicionTemperatura`, `MedicionPeso`) en un diccionario plano con números flotantes. Esto permite que la `VistaReporte` solo necesite saber cómo leer llaves de un diccionario y colocar los números en las columnas de una tabla visual.